

ALFIDO TECH

Cloud Computing Internship

TASK 3 COMPLETION REPORT

Create a Serverless Function (AWS Lambda / Cloud Function)

Submitted By	Saniya Shaikh
AWS Account	785446018888
Region	US East-1 (N. Virginia)
Function Name	MyFunction
Function ARN	arn:aws:lambda:us-east-1:785446018888:function:MyFunction
Runtime	Python 3.12
API Gateway	MyFunction-API
Public Endpoint	https://3r1yc5udya.execute-api.us-east-1.amazonaws.com/default/MyFunction
Test Event	myTestEvent (Synchronous)
Date	April 2026
Status	☒ COMPLETED

Step 1. Create Lambda Function

A new AWS Lambda function named MyFunction was created in the US East-1 region using the Python 3.12 runtime. The function was successfully provisioned with its Function ARN (arn:aws:lambda:us-east-1:785446018888:function:MyFunction) and the code editor opened automatically with the lambda_function.py file ready for editing.

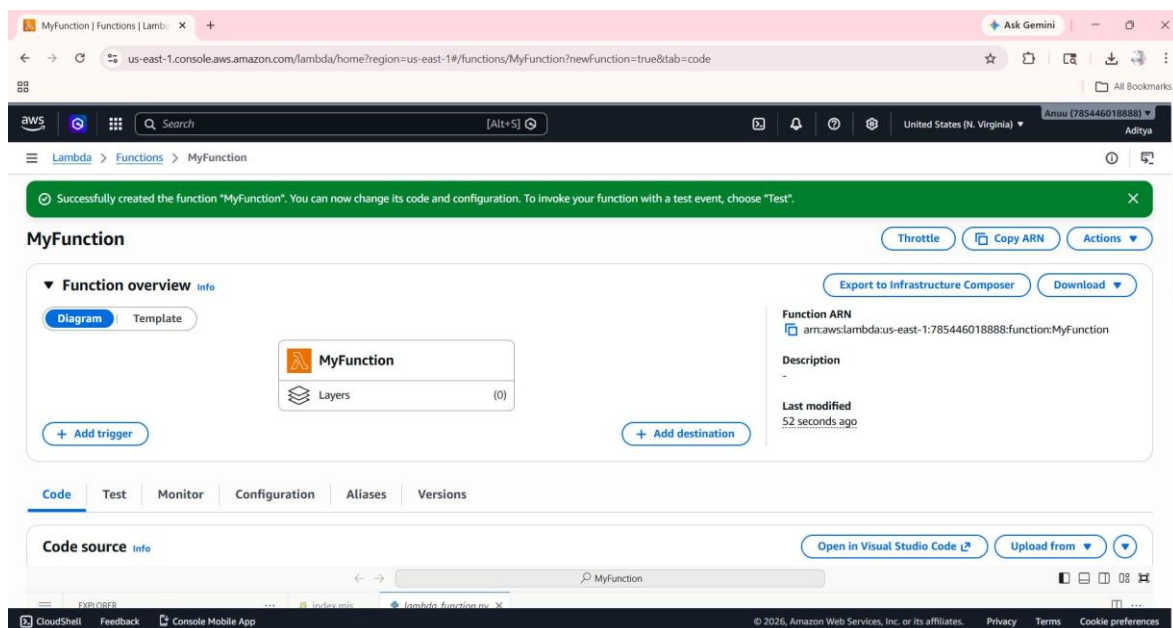


Figure 1 – Lambda function 'MyFunction' successfully created | ARN: arn:aws:lambda:us-east-1:785446018888:function:MyFunction

Step 2. Write & Deploy Function Code (Python 3.12)

The Lambda function code was written in Python 3.12 inside lambda_function.py. The function returns a JSON response with statusCode 200, CORS headers, and a structured body containing the message, task, status, runtime, platform, and intern fields. The function was deployed using the Deploy button (Ctrl+Shift+U).

Function code (lambda_function.py):

```
import json
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {
```

```

        "Content-Type": "application/json",
        "Access-Control-Allow-Origin": "*"
    },
    "body": json.dumps({
        "message": "Hello from AWS Lambda!",
        "task": "Alfido Tech - Task 3",
        "status": "success",
        "runtime": "Python 3.12",
        "platform": "AWS Lambda (Serverless)",
        "intern": "Saniya"
    })
}

```

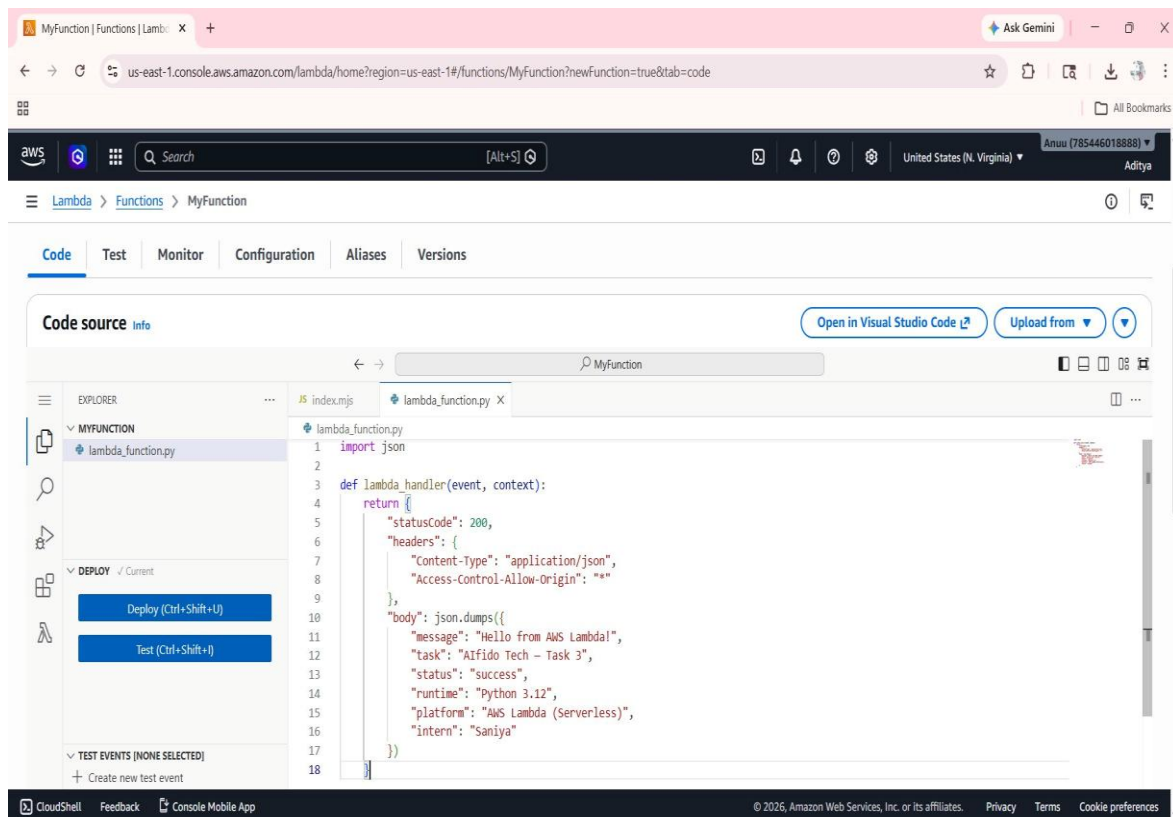


Figure 2 – Lambda function code in the inline editor (lambda_function.py) — Python 3.12 runtime

Step 3. Test Function via Cloud Console

A test event named myTestEvent was created using the Synchronous invocation type. The function was executed via the Test tab and the console confirmed 'Executing function: succeeded'.

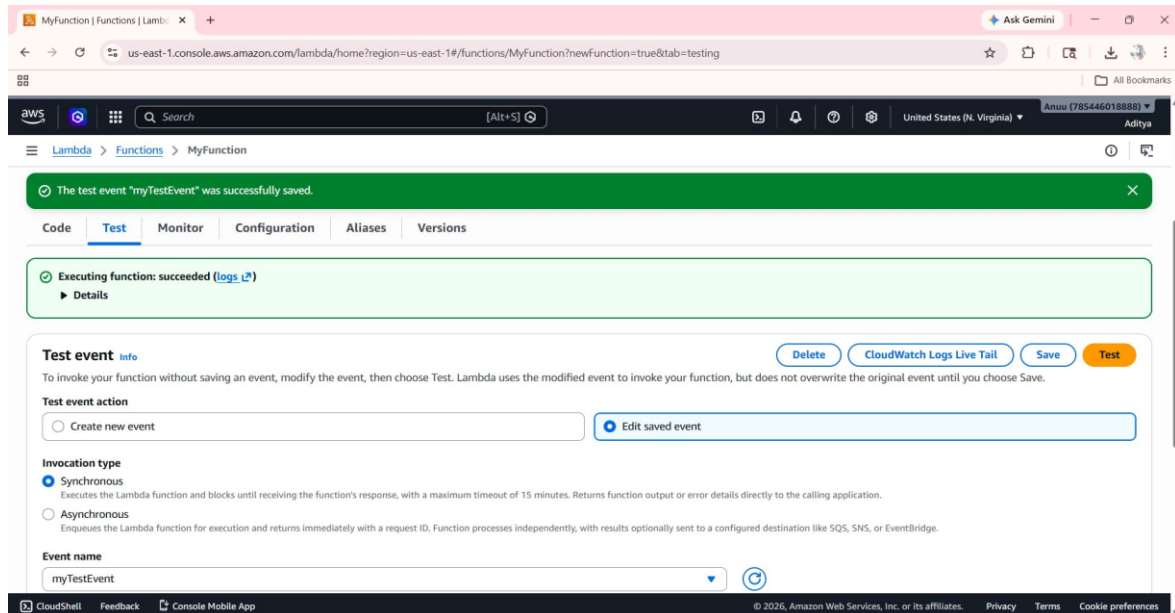


Figure 3 – Test event 'myTestEvent' executed successfully | 'Executing function: succeeded' confirmed in Lambda console

Step 4. Deploy + Generate Public Endpoint (API Gateway)

An API Gateway trigger named MyFunction-API was successfully added to the Lambda function. The trigger appears in the Function overview diagram and the Configuration → Triggers tab confirms the public API endpoint is active and accessible.

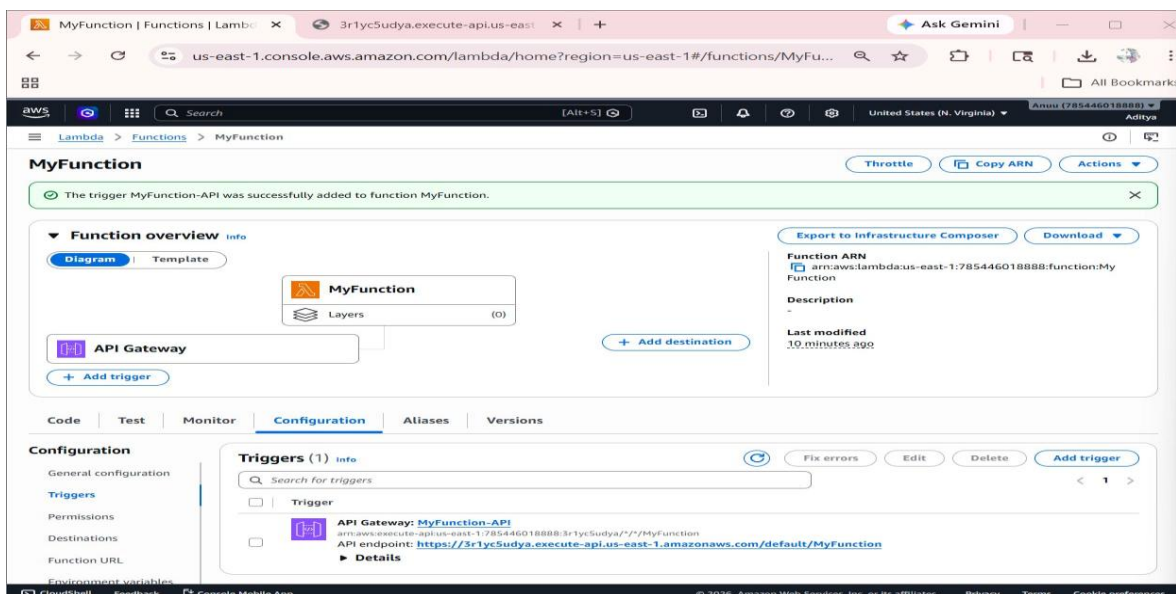


Figure 4 – API Gateway 'MyFunction-API' trigger successfully added | Public endpoint visible under Configuration → Triggers

Step 5. Live JSON Response via Public Endpoint

The public API Gateway endpoint was accessed in the browser and returned the expected JSON response with Pretty-print formatting enabled. The response confirms the function is live, serverless, and returning structured JSON data successfully.

JSON Response returned by the endpoint:

```
{
  "message": "Hello from AWS Lambda!",
  "task": "Alfido Tech - Task 3",
  "status": "success",
  "runtime": "Python 3.12",
  "platform": "AWS Lambda (Serverless)",
  "intern": "Saniya"
}
```

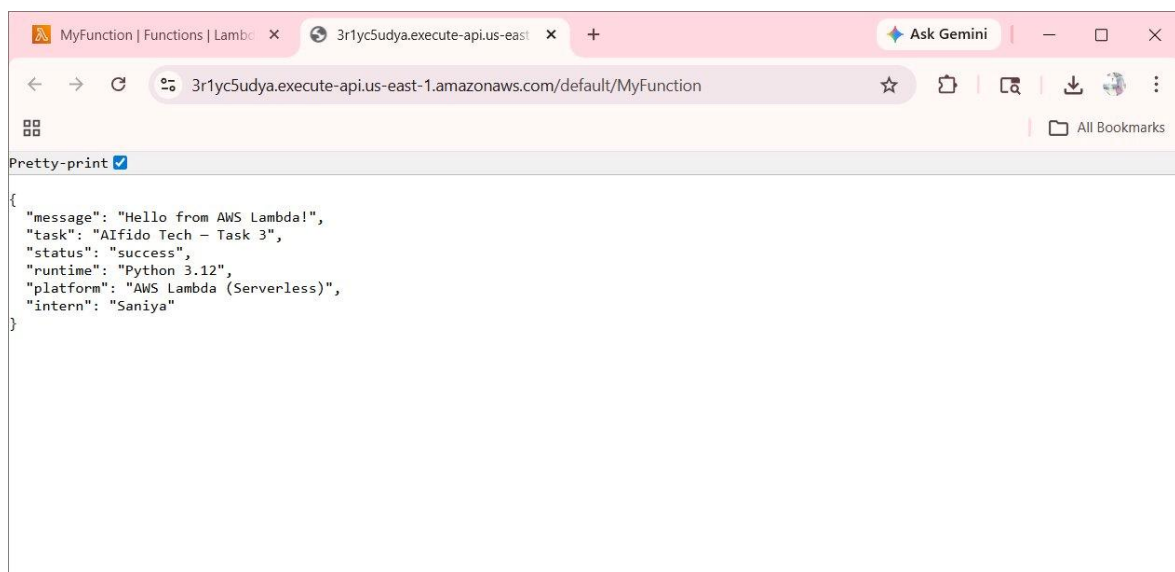


Figure 5 – Live JSON response at <https://3r1yc5udya.execute-api.us-east-1.amazonaws.com/default/MyFunction>

Task Completion Summary

Requirement	Action Taken	Status
Create function using Node.js / Python	Lambda function created with Python 3.12 runtime	✓ Done
Test function using cloud console	myTestEvent created; execution succeeded in Test tab	✓ Done
Deploy + generate public endpoint (API Gateway)	MyFunction-API trigger added; public URL generated	✓ Done
Function returns JSON output	Endpoint returns structured JSON with statusCode 200	✓ Done

Deliverables

1. Endpoint URL Returning JSON:

<https://3r1yc5udya.execute-api.us-east-1.amazonaws.com/default/MyFunction>

2. Code of Function:

- `lambda_function.py` — Python 3.12 (see Figure 2 & code block in Step 2)
- Returns JSON with statusCode 200, CORS headers, and structured body
- Function name: MyFunction | Runtime: Python 3.12

3. Screenshots Included:

- Lambda function created — MyFunction overview (Figure 1)
- Function code in inline editor — `lambda_function.py` (Figure 2)
- Test execution succeeded — myTestEvent (Figure 3)
- API Gateway trigger added — MyFunction-API with endpoint URL (Figure 4)
- Live JSON response in browser at public endpoint (Figure 5)